

AI wrote the Python. How do you know it works?

A testing and debugging cheat sheet
for AI-generated code.

Behaviour first

The test matrix

assert

pytest

Debug with evidence

AI failure modes

Minimal repro

A review prompt

Sagar Rathkanthiwar

Data & AI Professional

10 slides · save for later →

Write the answer before the code

A vague requirement gets vague code. Turn the sentence into **concrete inputs and the exact output you expect** — then hand that to the AI.



Those six lines are your **spec and your first test suite at the same time**. Notice how writing them forced three decisions the original sentence never made: country codes, blank values, and what “too short” means.

Six cases, every single function

Don't guess at what to test. Walk this list — it takes two minutes and catches most of what AI gets wrong.

1. Happy path

The normal case it was written for.
`clean_phone("425-555-0132")`

2. Empty input

Empty string, empty list, empty file, zero rows.
`clean_phone("") · summarize([])`

3. Wrong type

A number where text is expected, or None.
`clean_phone(None) ·
clean_phone(4255550132)`

4. Boundary value

The edge of the valid range, and one step past it.
`9 digits · 10 digits · 11 digits`

5. Missing data

NaN, None, blank cells, absent dictionary keys.
`row.get("phone") is None`

6. Large / realistic

Real volume and real mess — not three tidy rows.
`50,000 rows from yesterday's export`

AI code is usually excellent at case 1 and **silently improvised for cases 2–6**. That is exactly where your time is worth the most.

“It ran” is not “it is correct”

A script that finishes without an error has proved one thing: **nothing crashed**. An assertion is how you check the answer.

```
def clean_phone(raw):
    digits = "".join(c for c in str(raw) if c.isdigit())
    if len(digits) == 11 and digits.startswith("1"):
        digits = digits[1:]
    return digits if len(digits) == 10 else None

# An assertion states a fact. If the fact is false, Python stops.
assert clean_phone("(425) 555-0132") == "4255550132"
assert clean_phone("+1 425 555 0132") == "4255550132"
assert clean_phone("") is None
assert clean_phone("5550132") is None          # boundary: too short

# Put the actual value in the message - future-you will thank you
got = clean_phone("425.555.0132")
assert got == "4255550132", f"expected 4255550132, got {got!r}"
```

Run the file and nothing prints? That silence is the result — every stated fact held. Note `{got!r}`: `repr` shows the quotes and stray whitespace that plain printing hides. `assert` is stripped out when Python runs with `-O`, so keep it for tests, never for validating production input.

Graduate to `pytest`

Loose asserts get lost. Move them into a file named `test_*.py` and every check re-runs on demand, forever.

```
# test_phones.py          run it with:  pytest -q
import pytest
from phones import clean_phone

def test_strips_formatting_characters():
    raw = "(425) 555-0132"          # Arrange - set up the input
    result = clean_phone(raw)       # Act      - call the one thing
    assert result == "4255550132"   # Assert   - check the result

@pytest.mark.parametrize("raw, expected", [
    ("425-555-0132", "4255550132"), # happy path
    (" +1 425 555 0132", "4255550132"), # country code
    ("", None),                    # empty input
    ("5550132", None),             # boundary: too short
])
def test_clean_phone_cases(raw, expected):
    assert clean_phone(raw) == expected
```

- **Arrange → Act → Assert.** One input, one call, one claim. Two behaviours to check means two tests, not two asserts.
- **Name the test after the behaviour**, not the function.
`test_strips_formatting_characters` tells you what broke without opening the file.
- `parametrize` **turns your matrix into rows.** Add an edge case in one line; each row passes or fails on its own.

Read the error before you change the code

The traceback holds the answer. Read the **bottom line first** (what went wrong), then scan up to **the last line that is your file** (where it went wrong).

Traceback (most recent call last):

print()

Fastest first move. Print the value *and* its type:

```
print(repr(x), type(x))
```

— one line before the failure, one after.

logging

For code that runs unattended. A timestamped trail you can switch off with one config line — not prints you must remember to delete.

breakpoint()

Drop it one line above the failure and inspect every variable live.

```
n
```

next ·

```
c
```

continue ·

```
q
```

quit. Or

```
pytest --pdb
```

lands you exactly where a test failed.

Then change **one thing at a time** and re-run. Two edits at once and you no longer know which one fixed it — or which one broke something else.

What to look for on the first read

Invented input shape

It assumed a column name, a date format, a dictionary key, a delimiter. It had to guess — it never saw your data.

`row["amount"]` -> `KeyError` on the real file

Silent fallback values

A missing value quietly becomes

`0`

or an empty string. Nothing errors; your total is just wrong.

`float(row.get("amount", 0))`

Over-broad except

`except Exception: pass`

swallows typos, network failures and real bugs identically.

catch the specific error instead

Off-by-one logic

Ranges and slices that drop the last item, or a

`>`

that should be

`>=`

. Test the boundary, always.

`range(1, len(x))` skips index 0

Happy path only

No handling for empty input, a missing file, or a duplicate row — because the example it learned from had none.

works on 3 rows, dies on 30,000

Confidently stale

A deprecated argument or a library version that has moved on. It reads perfectly and no longer exists.

check the docs, not the confidence

None of these throw an obvious error on line one. **Four of the six produce a wrong answer with a clean exit code** — which is precisely why “it ran” cannot be your finish line.

Shrink it before you ask for help

Pasting 300 lines and “it doesn't work” gets you a guess. **Isolate the failing function with hard-coded input** and you often find the bug yourself — before you even hit send.

```
# repro.py - everything needed to reproduce, nothing else
from clean import summarize

rows = [                                # smallest input that still fails
    {"id": 1, "amount": "12.50"},        # <- text, not a number
    {"id": 2, "amount": 8},
]
print(summarize(rows))

# expected: 20.5
# actual:   TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- 1 Cut the input down** until one more deletion makes the bug disappear. Two rows beat two thousand.
- 2 Remove the surroundings** — the dashboard, the API call, the database. Can it fail in one file?
- 3 Write down expected vs. actual.** If you cannot state what you expected, that is the real bug.

A minimal reproducible example is the **highest-leverage debugging move there is** — and it doubles as your next regression test.

Make the AI attack its own code

Asking “is this correct?” gets you reassurance. Asking for **assumptions** and **failing tests** gets you evidence.

PASTE THIS ABOVE YOUR FUNCTION

Review this Python function. List the **assumptions** it makes about its input, the **edge cases** it does not handle, and the **failure modes** that would produce a wrong answer without raising an error.

Then write `pytest` **tests that would expose them** — including empty input, wrong type, boundary values and missing data.

Do not rewrite the function yet. **Show me the failing tests first.**

- **“Do not rewrite yet” is the load-bearing line.** It stops the model quietly patching over the bug so you never learn what it was.
- **Then run the tests yourself.** A generated test that has never been executed is a claim, not a check — and some of them will be wrong.
- **Keep the ones that fail.** Fix the code until they pass, commit them, and that bug can never come back unnoticed.

AI can generate code in seconds.

Confidence comes from evidence.

- Specify the **behaviour** before the code — inputs, outputs, and what counts as invalid.
- Run the **six-case matrix**: happy path, empty, wrong type, boundary, missing data, realistic volume.
- Turn every bug you fix into a **test that stays**. That is how regressions stop repeating.
- The model can write the tests. **Only you can decide what “correct” means** for your data and your users.

**What test has saved you from a bug recently?
Drop it in the comments — someone is shipping that bug today.**

Sagar Rathkanthiwar · Data & AI Professional · follow for more field guides

Note: examples target Python 3.8+ and `pytest` 7+. `assert` statements are removed when Python runs with the `-O` flag — use explicit checks and raised exceptions to validate production input, and keep `assert` for tests and development-time invariants.